CSDS 395 Project Proposal for Research Mapping Visualizer

Alexander Holden, Logan Stanchak, Seth Omeike, Henry Koyner, Mace Korytko

Overview

This project proposes the development of an interactive visualization platform designed to map and explore academic research papers. By representing research activity, citation networks, and trending disciplines as a dynamic landscape, the tool will allow users to navigate clusters of scientific literature spatially and intuitively. This project will combine a machine learning algorithm capable of clustering related papers and an intuitive, interactive software platform interface that displays how papers relate to one another and evolve across publication timelines.

Machine Learning Algorithm

In order to construct a spatial and dynamic landscape of academic literature, the machine learning pipeline is designed to transform high dimensional text and citation data into a low dimensional, clusterable metric space. The proposed machine learning architecture is as follows.

1. Representation Learning and Embedding Generation:

To capture the semantic meaning of research papers, the system will process titles, abstracts and full text (where available and feasible) using transformer based language models. These models convert unstructured text into vector representations that encode contextual relationships and thematic overlap between publications.

2. Dimensionality Reduction and Spatial Mapping

Raw transformer embeddings exist in high-dimensional spaces, so the pipeline will require non-linear dimensionality reduction optimized for large scale graph data. This process compresses vectors into 2D coordinates while preserving local neighborhoods and global structure.

3. Dynamic Clustering, Influence Scoring and Temporal Metadata

To group related papers into clusters, the system will apply a density based clustering algorithm and graph neural networks that factor in citation networks and co-authorship metadata. To highlight landmark studies, the pipeline will compute an influence score using bibliometric indicators such as citation velocity, total citation count, PageRank within the citation network,

etc. This score will be normalized and outputted as a scaling parameter for the visualization layer. Publication dates will be extracted as orthogonal metadata allowing 3D timeline support without distortion.

Software Interface

The output from the machine learning algorithm will then be fed to full-stack web service, where the data is visualized and displayed to the user.

The user interface will be designed with an interactive, web-based platform that will visualize the academic literature in a visible node landscape. To handle the high-performance rendering of the thousands of nodes (the research papers), and their edges (connections to other papers), the front-end will utilize a WebGL 3D Force-Directed Graph engine that is built upon Three.js, a javascript library that makes creating and displaying 3D graphics easier.

The client will feature a visual representation of the individual research papers, served as clickable nodes. The size of the nodes will scale dynamically based off its influence score (e.g. citation count, PageRank, etc.) which will be computed by our backend. Nodes will be color coded based on their specific academic discipline or by a thematic clustering algorithm determined by HDBSCAN.

The client will allow for users to pan, zoom, and rotate the graph smoothly in a three-dimensional space. Upon hovering or clicking a specific node, a modal will be rendered, detailing the metadata about the research paper.

In order for users to visualize how research has evolved over time, we will utilize the z-axis for for the timeline. The x and y axis will be used for the semantic relationships and citations of the paper but the z-axis will constrain the nodes to their publication date, which will transform the map into a chronological landscape.

1. Frontend

The frontend will be programmed using HTML, CSS, Typescript, and React. We will also utilize the 3d-force-graph library, which will help us render a 3d-space that contains a set of nodes and edges that will be outputted by our machine learning algorithm, sizing nodes based on their influence score. The library includes support for camera controls relative to the graph, and accepts a plain JSON object of {nodes: [], links: []} from the backend service.

2. Backend:
The ML team will extract the data from our collection of academic papers and generate the raw embeddings of them. Then the ML team will perform clustering on these embeddings and creates an API with an interface for reading these clusters. The frontend will read the data from

the interface and render it in the client. We will perform tests to verify that everything is working smoothly before increasing the size of our graph.

3. Data

The papers and related metadata will be sourced from the Kaggle Dataset linked below. This dataset contains all the metadata required for the fronted, curated summaries and paper abstracts for context clustering and unique arXiv identifiers for arXiv API calls if full paper downloads are necessary.

https://www.kaggle.com/datasets/sumitm004/arxiv-scientific-research-papers-dataset

This dataset will be used to train the machine learning algorithms and provide the necessary metadata for the software interface.

Attaining citations requires querying the Semantic Scholar API which contains the required data.

https://www.semanticscholar.org/product/api

Necessary Background and Requirements

Machine Learning Algorithms and Requirements:

- SciBERT ~ pretrained language model created to process scientific text
    - Frameworks/Libraries: HuggingFace transformers, PyTorch, sentence-transformers
    - https://github.com/allenai/scibert/
    - https://arxiv.org/abs/1903.10676
- Uniform Manifold Approximation and Projection (UMAP) or t-SNE
    - Frameworks/Libraries: umap-learn
    - https://umap-learn.readthedocs.io/en/latest/
- Hierarchical Density-Based Spatial Clustering of Applications with Noise (HBDSCAN)
    - Frameworks/Libraries: hdbscan
    - https://hdbscan.readthedocs.io/en/latest/how_hdbscan_works.html
- Graph Neural Network (GNN)
    - Frameworks/Libraries: scikit-learn, PyTorch Geometric
    - https://pytorch-geometric.readthedocs.io/en/latest/
- PageRank algorithm adapted for citation networks
    - Frameworks/Libraries: NetworkX
    - https://networkx.org/en/
    - https://www.geeksforgeeks.org/python/page-rank-algorithm-implementation/

Software Interface Requirements:

- 3D Force-Directed Graph
    - Three.js, 3d-force-graph
- Python API
- HTML, CSS, TS, React

Management Plan and Timeline

Week 1-3:
- Brainstorming and proposal

Week 4:
- Set up shared repo
- Dependency management:
- Scrape the bounded initial dataset (one or two small subfields to start)
    - Use the ArXiv API as opposed to scraping the full site
- Store metadata into a database
- Scaffold the front end

Week 5-6:
- Generate SciBERT embeddings → run UMAP → run HDBSCAN, spot-check cluster coherence
- Build real endpoints, citation graph ingestion in NetworkX, wire up auth/CORS/etc.
- Frontend: node coloring by cluster, sizing by influence score, camera controls, hover/click modal — all built against mock data

Week 7
- Compute pagerank and citation velocity, normalize influence scores
- Incorporate real ML output via API
- Fully connect frontend to backend, fix any schema mismatches

Week 8
- Finalize publication date metadata formatting
- Incorporate z-coordinates for publication date
- Implement preliminary z-axis in visualization

Week 9
- Retune parameters and model based on the visualizations
- Performance tuning for more papers
- Polish frontend
- Demo due

Week 10
- Expand data test
- Stress test frontend and backend

Week 11

- Integration testing
- UI/UX pass
- Edge case handling

Week 12

- Final demo prep
- Documentation
- Write-up
- Presentation